IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins

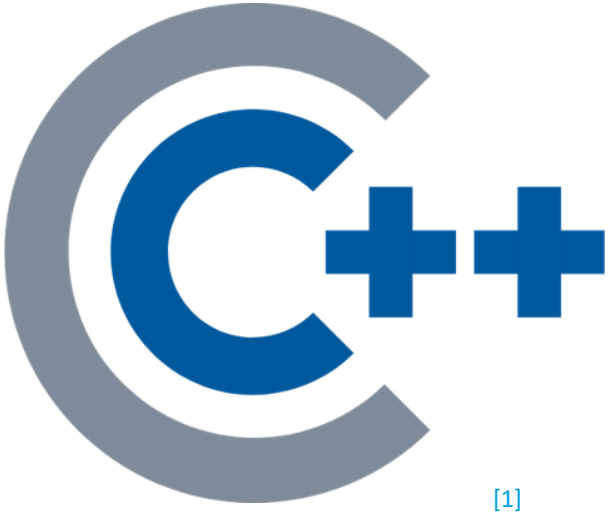
UFRN | metrópole DIGITAL

AULA 09

Programação em Python para Microcontroladores

Introdução – O que é Python?

- Como vimos em sala de aula, existem diversas linguagens de programação para microcontroladores.
- A principal utilizada até agora foi o C/C++, bastante eficiente e próxima do hardware.
- No entanto, ela não é a única opção.
- Outras linguagens também podem ser usadas, como o Python, ou mais especificamente, o MicroPython, uma versão adaptada para rodar em microcontroladores.



[1]

Introdução – O que é Python?

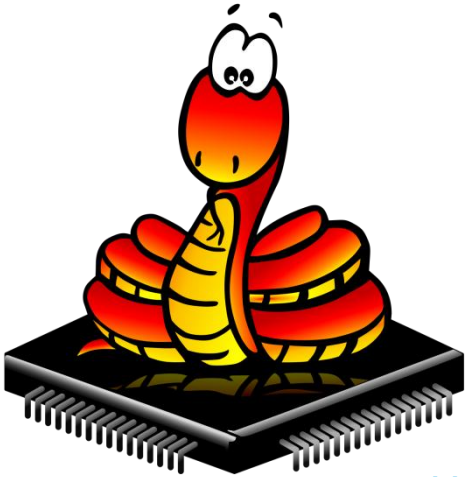
- Antes de tudo, o que seria o Python?
  - Linguagem de alto nível e interpretada
    - Código mais próximo da linguagem humana.
    - Executado linha por linha por um interpretador, no modelo REPL (Read, Eval, Print, Loop):
      - Permite escrever e testar comandos de forma interativa, direto no terminal.
  - Multiparadigma, suportando diferentes tipos de programação:
    - Estruturada, imperativa, orientada a objetos, funcional.
  - Ampla aplicação em diversas áreas:
    - Ciência de dados
    - Desenvolvimento Web
    - Automação
    - Inteligência Artificial
    - Robótica
    - Sistemas embarcados (via MicroPython)



[2]

Introdução – O que é MicroPython?

- Mas e o MicroPython?
  - Implementação enxuta do Python 3, desenvolvida para rodar em microcontroladores e sistemas embarcados com recursos limitados (memória e processamento).
  - Criado por Damien George em 2013.
  - Inclui:
    - Interpretador Python embutido em firmware, REPL interativo e uma seleção de bibliotecas adaptadas para lidar com pinos, sensores, comunicação serial e etc.
  - Código diretamente no dispositivo
    - Testar e executar código diretamente no microcontrolador.



[3]

Introdução – O que é MicroPython?

- Consumo de recursos:
  - 256 kB de memória flash e 16 kB de memória RAM.
- Bibliotecas limitadas:
  - Inclui apenas um subconjunto das bibliotecas do Python, além das bibliotecas específicas para Hardware.
  - Site Oficial das bibliotecas.
- Controle direto do hardware
  - Inclui módulos como machine e time que permitem controlar GPIOs, PWM, ADC, I2C, entre outros.



[4]

Introdução – O que é MicroPython?



[4]

- Microcontroladores suportados:
  - ESP32 e ESP8266 (limitado).
  - Raspberry PI Pico(RP2040, nativo).
  - STM32.
- IDEs:
  - Thonny, uPyCraft, VSCode com plugins.
- Instalação do firmware MicroPython
  - Baixar o firmware apropriado para a placa que se deseja programar:
    - Página dos firmwares
  - Gravar usando a ferramenta esptool da espressif (ESP-IDF ou a versão do arduino), mas se é a primeira vez, recomendado apagar a memória flash antes (esptool.py erase_flash).

Introdução – O que é MicroPython?

- Para que o microcontrolador execute scripts automaticamente:
  - boot.py
    - Executado sempre que o microcontrolador é inicializado.
    - Ideal para a configuração de rede, pinos e etc.
  - main.py
    - Executa após o boot.py.
    - Contém a lógica principal do programa.
  - Fazendo uma analogia, o arquivo boot seria o equivalente ao setup do Arduino e o arquivo main seria equivalente ao loop.
  - O nome dos arquivos devem ser exatamente esses sob pena do programa não rodar adequadamente.

Introdução – O que é MicroPython?

- Exemplo de código

Blink_uPython2 > boot.py > ...

```
1   from machine import Pin
2
3   led = Pin(27, Pin.OUT)
4   led.value(0)
5
```

[5]

Blink_uPython2 > main.py > ...

```
1   import time
2   from machine import Pin
3
4   led = Pin(27)
5
6   while True:
7       led.toggle()
8       print("led:")
9       print(led.value())
10      time.sleep(1)
11
```

[5]

Introdução – O que é MicroPython?

- Lista de comandos em MicroPython:
  - Modulo machine:
    - Pin, ADC, PWM, I2C, SPI, UART, Timer, reset.
      - Pin(numPin, tipo) – define o pino, se é saída, entrada, pull-up, pull-down.
      - .value() – define uma escrita ou leitura de um pino.
      - PWM(Pin()) – define que um pino será PWM.
      - .freq(x) – define a frequencia x em Hz.
      - .duty_u16(y) – define o ciclo de trabalho em um número unit_16
      - ADC(Pin()) – define o pino do ADC
      - .read_u16() – leitura do adc em um uint_16.
  - Modulo time
    - .sleep(z) – espera z segundos.
    - .ticks_ms() – retorna o tempo em milisegundos desde o boot.
    - .ticks_diff(a,b) – retorna a diferença entre dois ticks.

Introdução – O que é MicroPython?

- Limitações
  - É interpretado, o que o torna mais lento que programas compilados em C/C++.
  - Operações de tempo crítico podem não ter desempenho suficiente.
  - O garbage collector pode causar pausas imprevisíveis.
  - Não suporta todas as bibliotecas tradicionais.

Exercício

- Implementem o blink em micropython
  - Baixe e instale o firmware
  - Crie os arquivos boot e main
  - Grave os arquivos no microcontrolador e verifique seu funcionamento.

Referências complementares

- GEORGE ROBOTICS PTY LTD. MicroPython. 2025. Disponível em: https://micropython.org/. Acesso em: 15 abr. 2026.
- MICROPYTHON AUTHORS AND CONTRIBUTORS. MicroPython libraries. 2026. Disponível em: https://docs.micropython.org/en/latest/library/index.html. Acesso em: 15 abr. 2026.
- RANDOM NERD TUTORIALS. Install uPyCraft IDE – Windows PC Instructions. 2018. Disponível em: https://randomnerdtutorials.com/install-upycraft-ide-windows-pc-instructions/. Acesso em: 15 abr. 2026.

Referências complementares

- ODDY, Chris. MicroPython vs Python – What’s the Difference?. KO2 Recruitment, [s.d.]. Disponível em: https://www.ko2.co.uk/micropython-vs-python/. Acesso em: 15 abr. 2026.

Fontes

[1] – https://c-cpp-notes.vercel.app/docs/basics
[2] – https://commons.wikimedia.org/wiki/File:Python.svg
[3] – https://en.wikipedia.org/wiki/File:Micropython-logo.svg
[4] – https://en.m.wikipedia.org/wiki/File%3AMicroPython_new_logo.svg
[5] – Autoria própria

IMD0904 –Plataforma de Hardware para Internet das Coisas

Prof. Daniel Lopes Martins
daniel.martins@imd.ufrn.br

UFRN | metrópole DIGITAL